

BLOC06.

Bloc06Ejer01

Código

```
import java.util.ArrayList;
import java.util.List;
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;
public class Bloc06Ejer01 {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        List<String> listaPalabras = new ArrayList<String>();
        Map<Character,Integer>cantidadPrimeraLetra = new HashMap<>();
        listaPalabras = palabras();
        cantidadPrimeraLetra = cantidadPrimeraLetra(listaPalabras);
        System.out.println("-----Palabras Introducidas-----\n");
        for(String palabra:listaPalabras){
            System.out.println(palabra);
        }
        System.out.println("\n-----Cantidad de primeras letras-----\n");
        for(Character letra:cantidadPrimeraLetra.keySet() ){
            System.out.println(letra + " = " +cantidadPrimeraLetra.get(letra));
        }
    }

    public static List<String> palabras(){
        Scanner in = new Scanner(System.in);
        List<String> listaPalabras = new ArrayList<String>();
        boolean bucle = true;
        while(bucle){
            String finalizar = "finalizar";
            System.out.println("Introduce una palabra");
            String palabra = in.nextLine().trim();
            if(palabra.equals(finalizar)){
                return listaPalabras;
            }
            else if(!palabra.matches("^(?!\\s)\\S+$")){
                System.out.println("No se puede introducir espacios en blanco");
            }
            else if(palabra.matches("[0-9].*")){
                System.out.println("No puede empezar con un número");
            }
        }
    }
}
```

```

    }
    else{
        listaPalabras.add(palabra);
    }
}
return listaPalabras;
}

public static Map<Character, Integer> cantidadPrimeraLetra(List<String>listaPalabras){
    Map<Character,Integer>cantidadPrimeraLetra = new HashMap<>();
    char primeraLetra;
    for(String palabra:listaPalabras){
        primeraLetra = palabra.charAt(0);
        cantidadPrimeraLetra.put(primeraLetra,
cantidadPrimeraLetra.getOrDefault(primeraLetra, 0)+1);
    }
    return cantidadPrimeraLetra;
}
}

```

Capturas

```

--- exec:3.1.0:exec (default-cli) @ Bloc06Ejer01 ---
Picked up JAVA_TOOL_OPTIONS: -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
Introduce una palabra
Tomeu
Introduce una palabra
Andreu
Introduce una palabra
Jordi
Introduce una palabra
Bernat
Introduce una palabra
Bernat
Introduce una palabra
Andreu
Introduce una palabra
finalizar
-----Palabras Introducidas-----

Tomeu
Andreu
Jordi
Bernat
Bernat
Andreu

-----Cantidad de primeras letras-----

A = 1
Á = 1
B = 2
T = 1
J = 1
-----

```

```

Picked up JAVA_TOOL_OPTIONS: -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF
Introduce una palabra
Bernat
Introduce una palabra
Bernat
Introduce una palabra

No se puede introducir espacios en blanco
Introduce una palabra

No se puede introducir espacios en blanco
Introduce una palabra
Tomeu
Introduce una palabra
bernat
Introduce una palabra
finalizar
-----Palabras Introducidas-----

Bernat
Bernat
Tomeu
bernat

-----Cantidad de primeras letras-----

B = 2
b = 1
T = 1

```

```

--- exec:3.1.0:exec (default-cli) @ Bloc06Ejer
Picked up JAVA_TOOL_OPTIONS: -Dsun.stdout.enco
Introduce una palabra
FINALIZAR
Introduce una palabra
finalizar
-----Palabras Introducidas-----

FINALIZAR

-----Cantidad de primeras letras-----

F = 1

-----
BUILD SUCCESS
-----

Total time:  11.760 s
Finished at: 2026-03-09T19:48:22+01:00
-----

```

ContrassenyesLopez

Código

```
import java.util.Scanner;
import java.util.HashSet;
import java.util.Set;
import java.util.Random;
import java.util.InputMismatchException;
public class ContrassenyesLopez {

    private static final int MAX_INTENTOS_POR_CONTRASENA = 10;

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int numeroContraseñas = 0;
        int longitudContraseñas = 0;
        String contraseña;
        boolean datoCorrecto = true;
        Set<String> contraseñas = new HashSet<>();
        while(datoCorrecto){
            System.out.println("¿Cuántas contraseñas quieres crear?");
            try{
                numeroContraseñas = in.nextInt();
                in.nextLine();
                datoCorrecto = false;
            }
            catch(NumberFormatException e){
                System.out.println("Entrada no válida. Debes escribir un número entero. Inténtalo de nuevo.");
            }
        }
        datoCorrecto = true;
        while(datoCorrecto){
            System.out.println("¿Cuántos caracteres tiene que tener las contraseñas?");
            try{
                longitudContraseñas = in.nextInt();
                in.nextLine();
                datoCorrecto = false;
            }
            catch(NumberFormatException e){
                System.out.println("Entrada no válida. Debes escribir un número entero. Inténtalo de nuevo.");
            }
        }
        int colisiones = 0;
        int totalIntentos = 0;
```

```

boolean falloPorIntentos = false;
for (int i = 0; i < numeroContraseñas; i++) {
    boolean agregada = false;
    for (int intento = 1; intento <= MAX_INTENTOS_POR_CONTRASEÑA; intento++) {
        contraseña = creacionContraseña(longitudContraseñas);
        totalIntentos++;

        if (contraseñas.add(contraseña)) {
            System.out.printf("(%d/%d)Añadida: %s%n",
                contraseñas.size(), numeroContraseñas, contraseña);
            agregada = true;
            break;
        } else {
            colisiones++;
            System.out.printf("(%d/%d)Repetida (intento %d): %s%n",
                Math.min(contraseñas.size() + 1, numeroContraseñas),
                numeroContraseñas, intento, contraseña);
        }
    }
    if (!agregada) {
        System.out.println("\n No se han podido generar todas las contraseñas solicitadas
(se alcanzó el máximo de 10 intentos por contraseña).");
        System.out.printf("Se generaron %d de %d contraseñas solicitadas.%n",
            contraseñas.size(), numeroContraseñas);
        falloPorIntentos = true;
        break;
    }
    if (contraseñas.size() == numeroContraseñas) {
        break;
    }
}
if (!falloPorIntentos && contraseñas.size() == numeroContraseñas) {
    System.out.println("\n=== Contraseñas generadas ===");
    for (String c : contraseñas) {
        System.out.println(" - " + c);
    }

    double porcentajeColisiones = (totalIntentos == 0)
        ? 0.0
        : (colisiones * 100.0 / totalIntentos);

    System.out.printf("Duplicados: %d de %d intentos (%.2f%%)%n",
        colisiones, totalIntentos, porcentajeColisiones);
}
}

public static String creacionContraseña(int longitud){
    String contraseña;

```

```

Random random = new Random();
char[] password = new char[longitud];
for (int j = 0; j < longitud; j++) {
    char c = (char) (random.nextInt(26) + 'a');
    if (random.nextBoolean()) {
        c = Character.toUpperCase(c);
    }
    if (random.nextBoolean()) {
        password[j] = c;
    } else {
        password[j] = (char) ('0' + random.nextInt(10));
    }
}
contraseña = new String(password);

return contraseña;
}
}

```

Capturas

```

1 --- execrs.1.0:exec (default-cgi) @ Contrasenyeslopez --
Picked up JAVA_TOOL_OPTIONS: -Dsun.stdout.encoding=UTF-8
¿Cuántas contraseñas quieres crear?
5
¿Cuántos caracteres tiene que tener las contraseñas?
3
(1/5)Añadida: Xl4
(2/5)Añadida: 8Lr
(3/5)Añadida: 0T0
(4/5)Añadida: lJ9
(5/5)Añadida: sA7

=== Contraseñas generadas ===
- lJ9
- sA7
- Xl4
- 0T0
- 8Lr
Duplicados: 0 de 5 intentos (0,00%)
-----
BUILD SUCCESS
-----
Total time: 11.293 s
Finished at: 2026-03-09T19:39:40+01:00
-----
|

```

Picked up JAVA_TOOL_OPTIONS: -Dsun.stdout.encoding=UTF-8 -Dsun.

¿Cuántas contraseñas quieres crear?

50

¿Cuántos caracteres tiene que tener las contraseñas?

10

(1/50) Añadida: 200JT5kZBS
(2/50) Añadida: 53X604G6M8
(3/50) Añadida: 6if35dHBO6
(4/50) Añadida: 54iEjt9C8x
(5/50) Añadida: FVBP8U4325
(6/50) Añadida: 7l5m0D0PHy
(7/50) Añadida: 5324lTla76
(8/50) Añadida: J4U3yq6W36
(9/50) Añadida: 1r581z0s18
(10/50) Añadida: q457V4jcuN
(11/50) Añadida: HP6eJw5g5h
(12/50) Añadida: 6t1p96F42R
(13/50) Añadida: 57unnuM662
(14/50) Añadida: 622X39fx3L
(15/50) Añadida: F851fUk19p
(16/50) Añadida: 1k2j72194A
(17/50) Añadida: F06fI06hl8
(18/50) Añadida: 463K5y229Z
(19/50) Añadida: 3hnQ39P6L0
(20/50) Añadida: A7FG4Sv326
(21/50) Añadida: 1M0R93K4W6
(22/50) Añadida: aXC8GV587g
(23/50) Añadida: MEP31981hf
(24/50) Añadida: 2C8PW15P4Q
(25/50) Añadida: k9bf7mj0ce
(26/50) Añadida: 8lkFLIX218
(27/50) Añadida: 059n32glPX
(28/50) Añadida: 7f4FCwNF0R

=== Contraseñas generadas ===

- 44U4794b56
- 092Jewf4hc
- 514xg9x629
- 463K5y229Z
- HP6eJw5g5h
- 622X39fx3L
- ZHT57yX2Wx
- 4K3K80t924
- 2C8PW15P4Q
- 1M0R93K4W6
- 217k369138
- 8zD82z50BS
- k9bf7mj0ce
- 9W65V4rQzg

Duplicados: 0 de 50 intentos (0,00%)

```

J --- exec:3.1.0:exec (default-cli) @ ContraseñasLopez:
Picked up JAVA_TOOL_OPTIONS: -Dsun.stdout.encoding=UTF
¿Cuántas contraseñas quieres crear?
37
¿Cuántos caracteres tiene que tener las contraseñas?
1
(1/37)Añadida: r
(2/37)Añadida: a
(3/37)Añadida: V
(4/37)Añadida: 2
(5/37)Añadida: 0
(6/37)Añadida: 6
(7/37)Añadida: p
(8/37)Añadida: E
(9/37)Añadida: c
(10/37)Añadida: n
(11/37)Añadida: u
(12/37)Añadida: T
(13/37)Añadida: 4
(14/37)Añadida: 5
(15/37)Añadida: t
(16/37)Añadida: 8
(17/37)Repetida (intento 1): 5
(17/37)Añadida: l
(18/37)Añadida: H
(19/37)Añadida: G
(20/37)Añadida: A
(21/37)Repetida (intento 1): n
(21/37)Añadida: I
(22/37)Añadida: g
(23/37)Repetida (intento 1): 5
(23/37)Repetida (intento 2): 5
(23/37)Añadida: i
(24/37)Añadida: K
(25/37)Repetida (intento 1): K
(25/37)Añadida: l
(26/37)Repetida (intento 1): r

- y
Duplicados: 32 de 69 intentos (46,38%)
-----
BTTT D SUCCESS

```

StockLopez

Código

Main

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

```

```

import java.util.Scanner;

public class StockLopez {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        List<Articulo> listaArticulos = new ArrayList<>();
        System.out.println("Introduce códigos de artículos. Si pones 000 terminará");
        while(true){
            System.out.println("Pon un código");
            String codigo = in.nextLine().trim();
            if(codigo.equals("000")){
                break;
            }
            if(codigo.isEmpty()){
                continue;
            }
            Articulo existente = buscarCodigo(listaArticulos,codigo);
            if(existente == null){
                System.out.print("Descripción para " + codigo + ": ");
                String descripcion = in.nextLine().trim();
                if(descripcion.isEmpty()){
                    descripcion = codigo;
                }
                Articulo nuevo = new Articulo(codigo,descripcion,1);
                listaArticulos.add(nuevo);
            }
            else{
                existente.setCantidad(existente.getCantidad()+1);
            }
        }
        Collections.sort(listaArticulos);
        System.out.println("\nArtículos ordenados por código (Z -> A):");
        for(Articulo a : listaArticulos){
            System.out.println(a);
        }
    }

    private static Articulo buscarCodigo(List<Articulo> listaArticulos, String codigo){
        for(Articulo a : listaArticulos){
            if(a.getCodigo() != null && a.getCodigo().equalsIgnoreCase(codigo)){
                return a;
            }
        }
        return null;
    }
}

```

Articulo

```
public class Articulo implements Comparable<Articulo>{
    private String codigo;
    private String descripcion;
    private int cantidad;

    public Articulo(){
    }
    public Articulo(String codigo, String descripcion, int cantidad){
        this.codigo = codigo;
        this.descripcion = descripcion;
        this.cantidad = cantidad;
    }

    public String getCodigo(){
        return codigo;
    }
    public void setCodigo(String codigo){
        this.codigo = codigo;
    }

    public String getDescripcion(){
        return descripcion;
    }
    public void setDescripcion(String descripcion){
        this.descripcion = descripcion;
    }

    public int getCantidad(){
        return cantidad;
    }
    public void setCantidad(int cantidad){
        this.cantidad = cantidad;
    }

    @Override
    public int compareTo(Articulo articulo){
        if (this.codigo == null && articulo.codigo == null) return 0;
        if (this.codigo == null) return 1;
        if (articulo.codigo == null) return -1;
        return articulo.codigo.compareToIgnoreCase(this.codigo);
    }

    @Override
    public String toString(){
        return String.format("%s | %s | %d", codigo, descripcion, cantidad);
    }
}
```

}

Capturas

```
--- exec:3.1.0:exec (default-cli) @ StockLopez ---
Picked up JAVA_TOOL_OPTIONS: -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
Introduce códigos de artículos. Si pones 000 terminará
Pon un código
AAA
Descripción para AAA: Caja
Pon un código
AAA
Pon un código
AAA
Pon un código
ZZZ
Descripción para ZZZ: Bolsa
Pon un código
ZZZ
Pon un código
QER
Descripción para QER: Java
Pon un código
PPP
Descripción para PPP: Manzana
Pon un código

Pon un código
000

Artículos ordenados por código (Z -> A):
ZZZ | Bolsa | 2
QER | Java | 1
PPP | Manzana | 1
AAA | Caja | 3

-----
BUILD SUCCESS
-----

Total time: 01:04 min
Finished at: 2026-03-09T19:45:08+01:00
```